

OE-EC803B

Big Data Analysis

Combined Module Notes

Modules 1 – 6

Semester VIII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 0: Java Programming & OOP Foundations	5
1 Java Programming & OOP Foundations	5
1.1 Object-Oriented Programming (OOP) Principles	5
1.2 Java Collections Framework	5
1.3 JVM Memory Management	6
2 Structured Query Language (SQL) & Relational Algebra	6
2.1 Logical Execution Order of an SQL Query	6
2.2 SQL Joins	7
2.3 Subqueries and Common Table Expressions (CTEs)	7
3 Linux Command Line & Unix Text Processing	8
3.1 Core File Manipulation Commands	8
3.2 Redirections and Pipelines	8
3.3 Unix Text Processing Tools	9
Module 1: Types of Digital Data	12
1 Types of Digital Data	12
1.1 Detailed Characteristics	12
2 Introduction to Big Data	13
2.1 The 5Vs of Big Data	13
2.2 Challenges of Traditional Database Systems (RDBMS)	14
3 Big Data Analytics	14
3.1 Types of Big Data Analytics	14
4 History and Architecture of Apache Hadoop	15
4.1 History of Hadoop	15
4.2 Core Components of Apache Hadoop	15
4.3 Unix Tools vs. Hadoop	15
5 Hadoop Streaming	16
5.1 Mechanism and Data Flow	16
6 Hadoop Ecosystem	17
6.1 Ecosystem Tools Overview	17
7 IBM Big Data Strategy and InfoSphere Tools	17
7.1 IBM Big Data Strategy	17
7.2 IBM InfoSphere BigInsights	18
7.3 IBM Big Sheets	18
Module 2: The Design of HDFS	22
1 The Design of HDFS	22

2	HDFS Concepts	22
2.1	Blocks in HDFS	22
2.2	NameNode, DataNode, and Secondary NameNode	23
3	HDFS Interfaces and Command Line Interface	23
3.1	HDFS CLI (Command Line Interface)	23
3.2	Hadoop File System Interfaces	24
4	HDFS Data Flow	24
4.1	Anatomy of a File Read	24
4.2	Anatomy of a File Write	25
5	Data Ingestion: Sqoop and Flume	25
5.1	Apache Sqoop	25
5.2	Apache Flume	25
6	Hadoop Archives (HAR)	26
7	Hadoop I/O: Compression, Serialization, and File Structures	26
7.1	Compression Codecs	26
7.2	Serialization	27
7.3	File-Based Data Structures	27

Module 3: Anatomy of a MapReduce Job Run 30

1	Anatomy of a MapReduce Job Run	30
1.1	YARN Runtime Job Execution Steps	30
2	Failures in YARN	31
2.1	Task Failure	31
2.2	NodeManager Failure	31
2.3	ResourceManager / ApplicationMaster Failure	31
3	Job Scheduling	32
4	Shuffle and Sort Phase	32
4.1	Map Side Execution Details	32
4.2	Reduce Side Execution Details	32
5	MapReduce Types and Formats	33
5.1	Input Splits and InputFormat	33
6	Advanced MapReduce Features	33
6.1	Counters	33
6.2	Joins in MapReduce	33

Module 4: Apache Pig 37

1	Apache Pig	37
1.1	Execution Modes and Components	37
1.2	Pig Latin Data Model	37

1.3	Core Processing Operators	37
2	Apache Hive	38
2.1	Hive Architecture	38
2.2	Managed vs. External Tables	38
2.3	Partitioning vs. Bucketing	39
3	Apache HBase	39
3.1	HBase Architecture and Components	39
4	IBM Big SQL	40
Module 5: R and Machine Learning in Big Data		43
1	R and Machine Learning in Big Data	43
2	Supervised Learning Algorithms	43
2.1	Linear Regression	43
2.2	Logistic Regression	44
2.3	Decision Trees	44
2.4	Naive Bayes Classification	44
3	Unsupervised Learning Algorithms	45
3.1	K-Means Clustering	45
3.2	Association Rule Mining: Apriori Algorithm	45
4	Collaborative Filtering	46
5	Big Data Analytics with BigR	46

Module 0

Java Programming & OOP Foundations

1. Java Programming & OOP Foundations

Distributed frameworks in the Hadoop Ecosystem are heavily built on Java. A strong grasp of Java's runtime environment and Object-Oriented Programming (OOP) paradigms is essential to understand HDFS mechanics and MapReduce workflows.

Java Virtual Machine (JVM)

The **Java Virtual Machine (JVM)** is an abstract computing machine that provides a runtime environment in which Java bytecode can be executed. It acts as an engine that drives the execution of Java code, converting bytecode to machine-specific instructions and managing system memory.

1.1 Object-Oriented Programming (OOP) Principles

Java uses four primary OOP pillars which map directly to distributed programming:

- **Encapsulation:** Bundling fields and methods within a single class and restricting direct access (using access modifiers like 'private', 'protected', 'public') to protect the internal state of objects.
- **Inheritance:** Allowing a class (subclass) to inherit fields and methods from another class (superclass). In Hadoop, custom mappers and reducers inherit from base API classes.
- **Polymorphism:** The ability of an object to take on many forms. Used in Hadoop when custom serialized types implement interface contracts (e.g., overriding the 'write()' and 'readFields()' methods of the 'Writable' interface).
- **Abstraction:** Hiding implementation details and showing only functional interfaces using 'abstract' classes and 'interfaces'.

1.2 Java Collections Framework

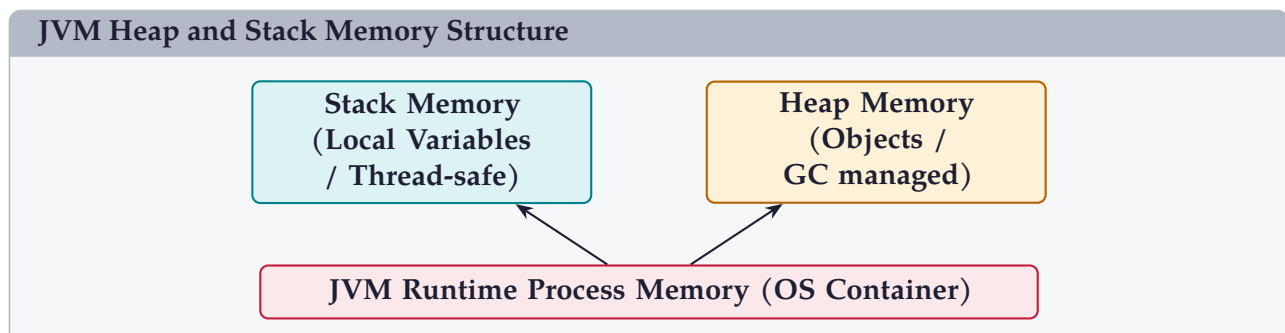
In MapReduce, intermediate values are held in collection data structures. The framework provides three primary categories of collections:

Collection	Key Characteristics	Underlying Structure	Hadoop Use Case
'ArrayList'	Ordered, indexable list. Allows duplicates. Resizes dynamically.	Resizable Array	Gathering list of values associated with a single key in Reducer.
'HashSet'	Unordered collection. No duplicates allowed.	Hash Table	Filtering out duplicate records or keys during processing.
'HashMap'	Key-value pairs storage. Unique keys. Fast search.	Hash Table + Linked List	Local in-memory caching and Map-Side Joins.

1.3 JVM Memory Management

The JVM splits its runtime memory into distinct regions. Understanding this is key to debugging Out-Of-Memory (OOM) errors in Hadoop:

- **Stack Memory:** Stores local variables and method execution frames. Allocation and deallocation are managed automatically.
- **Heap Memory:** Stores all objects instantiated via the 'new' keyword. Managed by the JVM Garbage Collector (GC). If MapReduce buffers grow too large, Heap memory exhaustion triggers a JVM crash.



2. Structured Query Language (SQL) & Relational Algebra

Data warehouse systems like Apache Hive use HiveQL, which is a dialect of SQL. Distributed querying translates SQL declarations into computational graphs.

Relational Databases vs. Hadoop Queries

Traditional databases use **Schema-on-Write**, validating data constraints during insertion. SQL engines on Hadoop (like Hive and Big SQL) use **Schema-on-Read**, overlaying structure on top of raw file streams (e.g., text, CSV, ORC) at query execution time.

2.1 Logical Execution Order of an SQL Query

While SQL queries are written starting with the 'SELECT' clause, the SQL compiler executes operations in a different logical sequence:

Logical SQL Execution Order

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

- **FROM/JOIN:** Identifies target datasets and links them.
- **WHERE:** Filters rows based on non-aggregated predicates.
- **GROUP BY:** Groups rows based on key values.
- **HAVING:** Filters groups using aggregate functions (e.g., 'HAVING count(*) > 5').
- **SELECT:** Projects fields and calculates functions.
- **ORDER BY:** Sorts output (very expensive in distributed systems as it forces global sorting).

2.2 SQL Joins

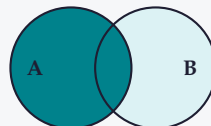
Joins are fundamental operations that merge datasets horizontally on a matching key column:

- **Inner Join:** Returns records that have matching values in both tables.
- **Left (Outer) Join:** Returns all records from the left table, and matched records from the right table. Fill with 'NULL' on the right side if no match.
- **Right (Outer) Join:** Returns all records from the right table, and matched records from the left table.
- **Full (Outer) Join:** Returns all records when there is a match in either left or right table.

SQL Joins Visual Representation



Inner Join



Left Join



Right Join

2.3 Subqueries and Common Table Expressions (CTEs)

- **Subquery:** A query nested inside another SQL statement. It can return scalar values, single columns, or tables (e.g., 'SELECT * FROM emp WHERE salary > (SELECT AVG(salary) FROM emp)').
- **Common Table Expression (CTE):** A temporary named result set defined using the 'WITH' keyword, which improves readability compared to nested subqueries:

Common Table Expression (CTE) Syntax

```
WITH DeptAvgSalary AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept_id
)
```

```
SELECT e.name, e.salary, d.avg_sal
FROM employees e
JOIN DeptAvgSalary d ON e.dept_id = d.dept_id;
```

3. Linux Command Line & Unix Text Processing

Hadoop CLI commands mimic standard Linux file system operations. Exposure to Unix text processing tools is essential for filtering, parsing, and verifying datasets before importing them.

Unix Streams

Unix systems model communications via three standard input/output channels (streams):

- **stdin (0):** Standard Input stream (default: keyboard).
- **stdout (1):** Standard Output stream (default: terminal screen).
- **stderr (2):** Standard Error stream (default: terminal screen).

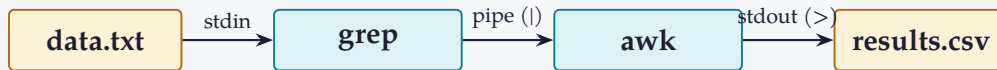
3.1 Core File Manipulation Commands

- **'cd <dir>':** Change current working directory.
- **'ls -la':** List directory contents showing file size, permissions, and hidden files.
- **'mkdir -p <path>':** Create directories recursively without throwing errors if they exist.
- **'cp -r <src> <dest>':** Copy files and directories recursively.
- **'mv <src> <dest>':** Move or rename files.
- **'rm -rf <path>':** Recursively force delete files or directories.
- **'chmod 755 <file>':** Modify file read-write-execute permissions (rwxr-xr-x).

3.2 Redirections and Pipelines

Commands can output data to files or other commands instead of printing to the terminal:

- **Redirections:** Redirect outputs or inputs to files.
 - **'>':** Redirects 'stdout' to a file (overwriting existing content).
 - **'>>':** Appends 'stdout' to a file.
 - **'2>':** Redirects 'stderr' to a file.
- **Pipes ('|'):** Passes the 'stdout' of one command as the 'stdin' of the next command.

Pipeline Data Flow

3.3 Unix Text Processing Tools

Hadoop Streaming allows using Unix programs as mappers and reducers. The core tools are:

- `'cat'`: Concatenates and displays file contents.
- `'grep <pattern>'`: Searches for regular expression pattern matches within files.
- `awk`: A pattern scanning and processing language. Heavily used to extract columns (e.g., `awk '{print $2}'` extracts the second space-separated field).
- `'sed'`: A stream editor for parsing and transforming text (e.g., substitute strings).
- `'sort'`: Sorts lines of text files alphabetically or numerically.
- `'uniq'`: Discards duplicate adjacent lines. Needs data to be pre-sorted.

Common UNIX Text Processing Pipeline

To find the top 5 most frequent error types from a log file:

```
cat server.log | grep "ERROR" | awk '{print $5}' | sort | uniq -c | sort -nr | head -n 5
```

Explanation: Step 1: Print file → Step 2: Filter lines containing "ERROR" → Step 3: Extract the fifth column → Step 4: Sort columns → Step 5: Count occurrences of unique elements → Step 6: Sort counts numerically in descending order → Step 7: Print top 5 lines.

Quick Revision — Module 0

Java Collections, SQL Queries & Unix Pipelines | OE-EC803B

One-Line Definitions

- **Polymorphism:** The OOP capability to invoke object-specific methods via interface abstractions.
- **Schema-on-Read:** A data schema validation approach where structure is applied only when reading files.
- **Common Table Expression (CTE):** A temporary named result set defined locally within an SQL query block.
- **Unix Pipe:** A mechanism that directs the standard output of one command to the standard input of another.
- **Garbage Collector (GC):** A background JVM process that automatically reclaims memory by deleting unreferenced objects.

Frequently Asked / Viva Questions

Q1. Why is Java memory management critical in Hadoop MapReduce jobs?

Custom mapper and reducer code instantiates objects inside JVM heap containers. If collections (like lists or hash maps) grow indefinitely without boundary checks, JVM Heap runs out of memory, leading to job failure.

Q2. Compare SQL's WHERE and HAVING clauses.

The 'WHERE' clause filters individual rows before grouping operations occur. The 'HAVING' clause filters aggregated groups after the 'GROUP BY' clause has been processed.

Q3. Explain the utility of SQL Subqueries vs. CTEs.

Subqueries are nested expressions that can make queries complex and hard to read. Common Table Expressions (CTEs) define clear, modular virtual tables using 'WITH', facilitating compiler optimization and improving readability.

Q4. What does the command "chmod 755 script.sh" accomplish?

It sets file permissions: Owner receives full permissions (Read=4, Write=2, Execute=1 → 7). Group and Others receive Read and Execute permissions (Read=4, Execute=1 → 5).

Q5. Why must "uniq" be preceded by a "sort" command in Unix pipelines?

The 'uniq' command only matches and filters out adjacent duplicate lines. If duplicate records are scattered, 'uniq' will miss them. Pre-sorting groups duplicate lines together.

Q6. What is the difference between ">" and ">>" in Unix Shell?

Frequently Asked / Viva Questions (contd.)

'>' redirects standard output to a file, overwriting the file's current contents. '>>' redirects output to a file, appending the output at the end of the file.

Q7. Describe the JVM Stack vs. Heap memory allocations.

Stack allocation is temporary, storing thread execution states and primitive local variables. Heap stores dynamic class objects that require garbage collection.

Q8. What is the time complexity of looking up a key in a HashMap?

On average, 'HashMap' lookup has $O(1)$ constant time complexity. In the worst case (with hash collisions), it defaults to $O(n)$ or $O(\log n)$.

Q9. Explain standard error (stderr) redirection in shell.

Errors bypass standard output redirection. To capture error logs, redirect standard error using '2>' (e.g., 'command > out.txt 2> error.log').

Q10. What is the difference between Schema-on-Write and Schema-on-Read?

Schema-on-Write enforces schema definitions when writing data to relational tables, rejecting formatting violations. Schema-on-Read stores raw data files as-is, parsing formatting layouts dynamically during queries.

Comparison Snapshot

Foundational Paradigm Comparisons:

Feature	SQL Database Engine	Unix Command Line
Execution Paradigm	Declarative (describe <i>what</i> data to fetch)	Procedural (pipe <i>how</i> data flows through tools)
Primary Objective	Structured relational database querying	General-purpose file processing and system control
Data Structures	Tables, columns, index structures	Text streams, byte buffers, files
Scaling Pattern	Relies on query planners and indexes	Relies on stream processing buffers
Optimization	Handled by query optimizer engines	Handled manually by pipeline construction

Module 1

Types of Digital Data

1. Types of Digital Data

Digital data represents any information stored, processed, or transmitted electronically. In modern computer systems, digital data is classified into three categories based on its structural characteristics:

Classification of Digital Data

Digital Data is categorized into three types:

1. **Structured Data:** Highly organized data conforming to a rigid pre-defined schema (e.g., relational databases).
2. **Semi-Structured Data:** Data containing internal identifiers, tags, or markers (no rigid schema, but self-describing; e.g., JSON, XML).
3. **Unstructured Data:** Data lacking any predefined structure or model (e.g., videos, PDF documents, raw text).

1.1 Detailed Characteristics

- **Structured Data:** Stored in tables with rows and columns. Uses Structured Query Language (SQL) for retrieval. It has fixed fields and schemas. *Examples:* Relational Databases (RDBMS), CSV files, spreadsheets.
- **Semi-Structured Data:** Does not conform to the formal structure of databases but contains tags or markers to separate data elements and enforce hierarchies. *Examples:* XML documents, JSON strings, HTML files, NoSQL Key-Value records.
- **Unstructured Data:** Represents the majority of real-world data (estimated at 80–90% of all enterprise data). It is difficult to store in standard tabular formats. *Examples:* MP3 files, MP4 videos, JPEG images, raw logs, emails, PDF manuals.

Parameter	Structured Data	Semi-Structured Data	Unstructured Data
Format	Tabular (Rows/Columns)	Tags/Key-value pairs	No specific structure
Schema	Schema-on-write (rigid)	Schema-on-read (flexible)	No schema applied
Storage	RDBMS (SQL Server, MySQL)	NoSQL databases, XML files	NoSQL databases, Data Lakes
Flexibility	Very low	Moderate	High
Scale	Hard to scale horizontally	Scalable horizontally	Highly scalable
Querying	Structured SQL queries	XQuery, JSON Path	Search indices, NLP, ML

2. Introduction to Big Data

Big Data

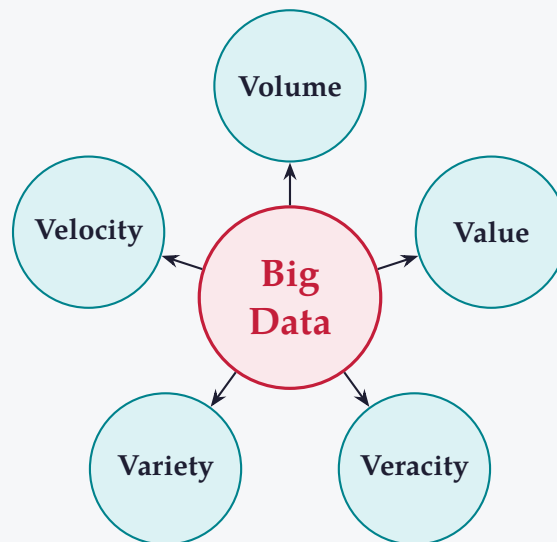
Big Data refers to datasets whose size (volume), complexity (variety), and rate of growth (velocity) are so large that traditional data processing application software (such as relational database management systems) are inadequate to capture, store, manage, and process them within a tolerable elapsed time.

2.1 The 5Vs of Big Data

The core dimensions defining Big Data are historically known as the **3Vs** (Volume, Velocity, Variety), which have been expanded to the **5Vs** to reflect reliability and business value:

1. **Volume:** The sheer scale of data generated. Measured in Terabytes (TB), Petabytes (PB), or Exabytes (EB).
2. **Velocity:** The speed at which new data is generated, transmitted, and processed in real-time (e.g., social media feeds, sensor streams).
3. **Variety:** The structural diversity of data types (structured, semi-structured, and unstructured data coming from various sources).
4. **Veracity:** The trustworthiness, quality, and accuracy of the incoming data (noise reduction, handling missing values).
5. **Value:** The actionable business insights extracted from the data, turning raw bits into commercial or operational intelligence.

The 5Vs of Big Data



2.2 Challenges of Traditional Database Systems (RDBMS)

Traditional RDBMS architectures fail to handle Big Data due to several fundamental limits:

- **Vertical Scaling (Scale-up) limits:** RDBMS scale by adding CPU, RAM, or storage to a single server. This has physical and cost limits. Big Data requires horizontal scaling (scaling out by adding cheap commodity nodes).
- **Rigid Schema Requirements:** RDBMS requires schema definitions before writing data. This prevents storing unstructured data or dynamic fields.
- **ACID Properties Overhead:** Locking rows/tables to maintain strict Consistency and Isolation creates severe latency spikes at high transaction speeds.

3. Big Data Analytics

Big Data Analytics

Big Data Analytics is the process of examining large and varied datasets to uncover hidden patterns, unknown correlations, market trends, customer preferences, and other useful information to make informed business decisions.

3.1 Types of Big Data Analytics

Analytics tasks are classified into four distinct stages based on complexity and value:

1. **Descriptive Analytics (What happened?):** Summarizes historical data to identify trends and patterns (e.g., standard reports, dashboards).
2. **Diagnostic Analytics (Why did it happen?):** Drill-down queries to identify the root cause of historical trends (e.g., analyzing customer churn).

3. **Predictive Analytics (What is likely to happen?):** Uses statistical models, forecasting algorithms, and machine learning to estimate future outcomes.
4. **Prescriptive Analytics (What should we do?):** Recommends action pathways based on simulations and optimization algorithms to capitalize on predictions.

4. History and Architecture of Apache Hadoop

4.1 History of Hadoop

Hadoop was created by Doug Cutting and Mike Cafarella in 2005. It originated from work on Nutch, an open-source web crawler project.

- It was inspired by Google's white papers on GFS (Google File System) published in 2003 and MapReduce framework published in 2004.
- Doug Cutting joined Yahoo! in 2006, and Yahoo! spun off Hadoop as an independent open-source project. In 2008, Apache Hadoop became a top-level project.
- Named after Doug Cutting's son's toy yellow elephant.

4.2 Core Components of Apache Hadoop

Hadoop provides a distributed software framework designed to run on clusters of commodity hardware.

Core Hadoop Modules

Hadoop's primary runtime architecture contains three core modules:

1. **HDFS (Hadoop Distributed File System):** The distributed storage layer that splits files into blocks and distributes them across cluster nodes.
2. **MapReduce:** The software programming framework for writing applications that process massive amounts of structured and unstructured data in parallel.
3. **YARN (Yet Another Resource Negotiator):** The cluster resource management layer that schedules jobs and allocates CPU/RAM resources across the cluster.

4.3 Unix Tools vs. Hadoop

Small datasets are processed locally using Unix utilities, whereas large datasets require distributed execution on Hadoop.

Parameter	Unix Utilities (grep, awk, sed)	Apache Hadoop
Execution Location	Single local machine	Distributed cluster (scale-out)
Data Volume Limit	Limited by local RAM and disk size	Petabytes (scales linearly with nodes)
Processing Paradigm	Sequential processing (single-threaded pipes)	Parallel, distributed execution (MapReduce)
Fault Tolerance	None (if the shell script crashes, run aborts)	High (data blocks replicated; jobs rerun automatically)
Data Locality	Reads data to CPU over system bus	Pushes computation code directly to storage nodes

5. Hadoop Streaming

Hadoop Streaming

Hadoop Streaming is a utility that comes bundled with the Hadoop distribution that allows users to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer (e.g., Python, Perl, Bash) by exchanging data via standard input ('stdin') and standard output ('stdout').

5.1 Mechanism and Data Flow

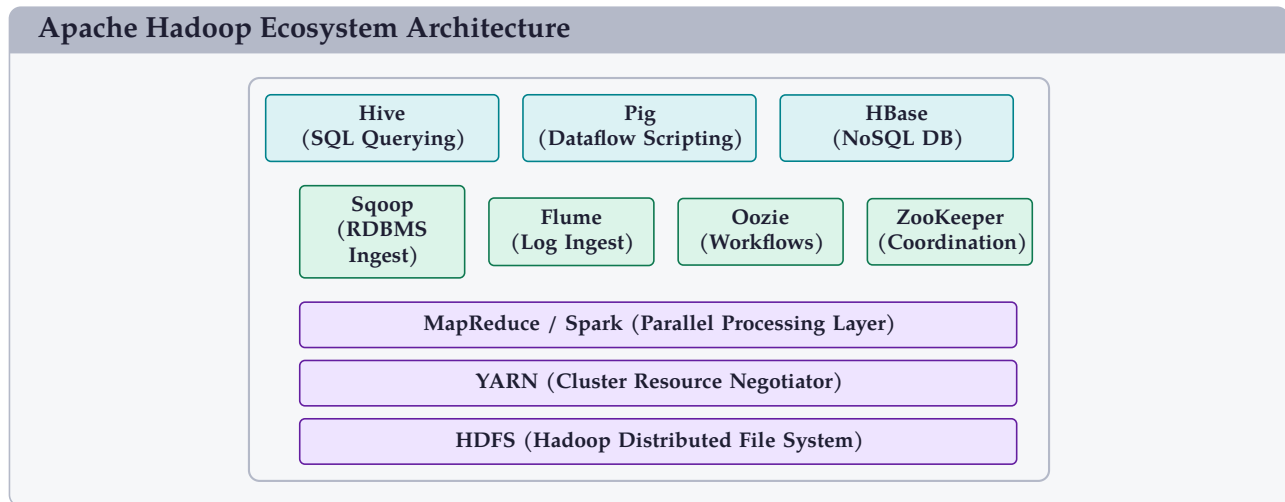
1. The Java MapReduce program spawns the mapper as a separate process and pipes HDFS input data line-by-line to the mapper process's 'stdin'.
2. The mapper program reads lines from 'stdin', processes the data, and writes tab-separated key-value pairs to its 'stdout'.
3. The streaming utility intercepts 'stdout' and handles sorting, partitioning, and shuffling to group keys together.
4. The streaming utility spawns the reducer process and pipes the grouped key-value data to its 'stdin'.
5. The reducer reads from 'stdin', aggregates values, and writes output key-value lines to 'stdout', which are captured and stored in HDFS.

Example: Python Mapper Script for Streaming

```
import sys
# Mapper: Reads lines from stdin, splits into words, outputs key-value
for line in sys.stdin:
    words = line.strip().split()
    for word in words:
        # Output: word (key) <TAB> 1 (value)
        print(f"{word}\t1")
```

6. Hadoop Ecosystem

The Hadoop framework is surrounded by a set of specialized helper tools that together form the **Hadoop Ecosystem**:



6.1 Ecosystem Tools Overview

- **Sqoop:** A command-line tool designed for transferring bulk data between Apache Hadoop and structured datastores such as relational databases (e.g., MySQL, Oracle).
- **Flume:** A distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of streaming log data into HDFS.
- **ZooKeeper:** A centralized service for maintaining configuration information, naming, providing distributed synchronization, and group services.
- **Oozie:** A workflow scheduler system to manage Apache Hadoop jobs.

7. IBM Big Data Strategy and InfoSphere Tools

7.1 IBM Big Data Strategy

IBM's Big Data Strategy focuses on integrating enterprise-grade reliability, security, and governance standards with raw open-source Apache Hadoop. It is built on three core pillars:

1. **Integration:** Bridging existing relational data warehouses (like IBM DB2) with unstructured Hadoop clusters.
2. **Governance:** Providing trace tools for tracking data lineage and enforcing access security.
3. **Acceleration:** Building analytical tools that execute queries faster than native MapReduce.

7.2 IBM InfoSphere BigInsights

InfoSphere BigInsights

IBM InfoSphere BigInsights was IBM's enterprise-grade distribution of Apache Hadoop. It combined the open-source Hadoop core with proprietary management tools, text analytics engines, and SQL querying features to simplify deployment and security.

Key features of BigInsights include:

- **SystemT:** A declarative, rule-based text analytics engine for extracting structure from unstructured documents.
- **Big SQL:** A low-latency, mass-parallel SQL querying engine that executes standard ANSI-SQL queries directly on HDFS.

7.3 IBM Big Sheets

Big Sheets

Big Sheets is a browser-based, spreadsheet-like visualization and analysis interface built on top of BigInsights that allows business analysts to manipulate and analyze large datasets without writing complex Java MapReduce or Pig scripts.

It utilizes a sheet model where users can apply formulas, filter rows, and build charts, which Big Sheets compiles into MapReduce jobs behind the scenes.

Quick Revision — Module 1

Introduction to Big Data & Hadoop Ecosystem | OE-EC803B

One-Line Definitions

- **Structured Data:** Highly formatted tabular data with a pre-defined schema (e.g., SQL tables).
- **Big Data:** Datasets whose scale, speed, and diversity exceed the limits of storage and CPU architectures.
- **HDFS:** A block-structured distributed file system designed for running on commodity hardware.
- **YARN:** YARN is the Resource Negotiator layer that manages execution scheduling and CPU/RAM allocation.
- **Hadoop Streaming:** A utility allowing mappers/reducers to run as independent scripts using stdin/stdout.
- **Big Sheets:** A web-based spreadsheet interface for visualizing and exploring Big Data on Hadoop.

Frequently Asked / Viva Questions

Q1. What are the differences between Semi-Structured and Unstructured data?

Semi-structured data lacks a formal database schema but contains internal tags or markers to represent structure (e.g., JSON, XML). Unstructured data has no predefined format or metadata identifiers (e.g., MP3 audio, video recordings, raw text).

Q2. Detail the 5Vs of Big Data and why they matter.

Volume (scale of data), Velocity (generation speed), Variety (data formats), Veracity (data reliability), and Value (operational insights). They define whether a dataset requires Big Data tools instead of traditional relational databases.

Q3. Why does RDBMS struggle with horizontal scaling?

RDBMS relies on maintaining ACID guarantees (Atomicity, Consistency, Isolation, Durability) across tables, which requires complex distributed locks. This creates severe performance bottlenecks when database tables are partitioned across different physical machines.

Q4. What Google technologies inspired the creation of Apache Hadoop?

Google File System (GFS) published in 2003 inspired the creation of Hadoop Distributed File System (HDFS), and Google's MapReduce paper published in 2004 inspired Hadoop's MapReduce framework.

Q5. How does Hadoop achieve data locality?

Frequently Asked / Viva Questions (contd.)

Hadoop schedules processing tasks directly on the physical node where the target HDFS data blocks are stored. This avoids copying large data files across the network, reducing network bandwidth bottlenecks.

Q6. What is the difference between vertical scaling and horizontal scaling?

Vertical scaling (scale-up) involves adding resources (CPU, RAM) to a single machine, which has clear hardware limits. Horizontal scaling (scale-out) involves linking together multiple cheaper servers (commodity nodes) in a cluster to act as one.

Q7. How does Hadoop Streaming handle communication between Java and other languages?

It spawns mappers/reducers as separate processes and communicates via standard I/O streams: input data is piped to the script's 'stdin', and the script writes output key-value pairs to its 'stdout'.

Q8. Explain the role of Sqoop and Flume in the Hadoop Ecosystem.

Sqoop is a tool used for bulk data transfer between relational databases (RDBMS) and HDFS. Flume is a distributed service designed for collecting and streaming log data into HDFS.

Q9. What is the purpose of ZooKeeper in Hadoop clusters?

ZooKeeper acts as a central coordinator for maintaining cluster configuration, managing leader elections (e.g., Active NameNode election in HA clusters), and synchronizing distributed locks.

Q10. What is IBM Big Sheets and who is its target user?

It is a web-based spreadsheet utility designed for business analysts to visualize and transform large datasets without writing MapReduce code. Behind the scenes, actions are converted into MapReduce tasks.

Comparison Snapshot

Traditional RDBMS vs. Apache Hadoop:

Feature	Traditional RDBMS	Apache Hadoop
Data Volume	Megabytes to Gigabytes (Gigabytes to Terabytes max)	Terabytes, Petabytes, and Exabytes
Data Variety	Only Structured (rigid tables)	Structured, Semi-Structured, and Unstructured
Schema Type	Schema-on-Write (rigorous validation)	Schema-on-Read (flexible query interpretation)
Access Paradigm	Interactive and Random (Low latency, high throughput)	Batch processing (High latency, massive scale throughput)
Scaling Type	Vertical (Adding bigger hardware)	Horizontal (Adding more commodity machines)
Cost per Gigabyte	High (requires enterprise servers and licensing)	Low (commodity hardware, open-source software)

Module 2

The Design of HDFS

1. The Design of HDFS

Hadoop Distributed File System (HDFS)

HDFS is a distributed, block-structured file system designed to store very large files (typically Gigabytes to Petabytes) reliably across a cluster of commodity servers, providing high throughput access to application data using a write-once, read-many access pattern.

Key design goals and assumptions include:

- **Commodity Hardware Failure:** HDFS assumes hardware failures are common rather than exceptional. High fault tolerance is built in through automatic block replication.
- **Large Datasets:** Optimized to support large files. A single HDFS cluster can support millions of files.
- **Streaming Data Access:** Designed for batch processing rather than low-latency interactive access. Focuses on high data throughput over low latency.
- **Simple Coherency Model:** Uses a **write-once, read-many** pattern. Once a file is written, closed, and replicated, it cannot be modified inline. Appends are supported in later versions but random edits are not.

2. HDFS Concepts

2.1 Blocks in HDFS

Unlike standard physical hard drives where disk blocks are 512 bytes or 4KB, HDFS files are divided into large, discrete chunks called **Blocks**.

HDFS Block Size Properties

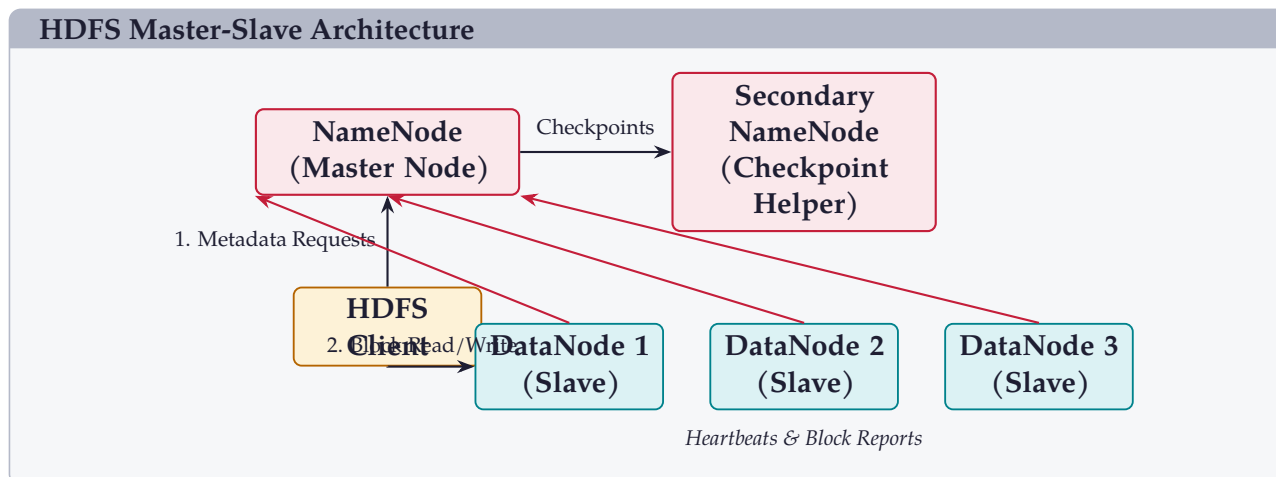
The standard block size in modern Hadoop distributions is **128MB** (or **256MB**).

- **Reason for large blocks:** To minimize the cost of disk seeks. By making blocks large, the time taken to transfer data from disk to network is significantly larger than the time taken to seek to the start of the block.

- **Block Storage:** If a file is smaller than the block size (e.g., a 10MB file), it does not consume the full 128MB of physical space; it only uses 10MB on the local disk.

2.2 NameNode, DataNode, and Secondary NameNode

HDFS uses a master-slave architecture containing three key operational nodes:



- **NameNode (Master):**
 - Manages the file system namespace (directories, files).
 - Tracks the mapping of files to block IDs and their physical DataNode locations.
 - Stores metadata in RAM for speed. Persists namespace changes in the **Edit Log** and holds the point-in-time filesystem state in **FSImage**.
- **DataNode (Slave):**
 - Stores and retrieves HDFS data blocks on the local physical filesystem as directed by the NameNode.
 - Sends periodic **Heartbeats** (confirming node availability) and **Block Reports** (listing all local blocks) to the NameNode.
- **Secondary NameNode:**
 - It is NOT a backup/failover NameNode.
 - It periodically merges the active **Edit Log** with the **FSImage** to prevent the Edit Log from growing too large. This process is called **Checkpointing**.

3. HDFS Interfaces and Command Line Interface

3.1 HDFS CLI (Command Line Interface)

Users interact with HDFS files using standard terminal commands prefixed with 'hdfs dfs' or 'hadoop fs'.

Common HDFS Terminal Commands

Command	Description and Syntax
'hdfs dfs -ls <path>'	Lists directory contents (analogous to Unix 'ls').
'hdfs dfs -put <src> <dest>'	Copies a local file from the client host into HDFS.
'hdfs dfs -get <src> <dest>'	Retrieves a file from HDFS and stores it locally.
'hdfs dfs -cat <path>'	Displays file contents directly on stdout.
'hdfs dfs -mkdir <path>'	Creates a new directory in HDFS.
'hdfs dfs -rm <path>'	Deletes a file or folder from HDFS.

3.2 Hadoop File System Interfaces

Hadoop supports programmatic interfaces to access HDFS files:

1. **Java API:** The native interface. Uses the 'org.apache.hadoop.fs.FileSystem' abstraction class, executing methods like 'open()', 'create()', and 'delete()'.
2. **WebHDFS REST API:** Provides complete read/write HTTP REST protocol endpoints to interact with HDFS from non-Java languages without loading fat native client libraries.

4. HDFS Data Flow

Data flow operations in HDFS isolate the client from the details of block replication and network routing.

4.1 Anatomy of a File Read

1. The Client calls 'open()' on the 'FileSystem' object (which is a 'DistributedFileSystem' instance).
2. The 'DistributedFileSystem' calls the NameNode using RPC (Remote Procedure Call) to get the locations of the blocks for the first part of the file.
3. The NameNode returns a list of DataNodes that have copies of those blocks, sorted by network proximity to the client.
4. The client calls 'read()' on the input stream ('FSDataInputStream'), which connects to the nearest DataNode.
5. Once the end of the block is reached, 'FSDataInputStream' closes the connection to that DataNode and opens a connection to the nearest DataNode for the next block, until reading is complete.

4.2 Anatomy of a File Write

1. The Client calls 'create()' on the 'DistributedFileSystem' object.
2. The 'DistributedFileSystem' makes an RPC call to the NameNode to create a new file in the filesystem namespace. The NameNode checks permissions and verifies the path does not already exist.
3. The client writes data to the output stream ('FSDataOutputStream'), which splits it into packets and writes them to an internal queue called the **Data Queue**.
4. The **Data Streamer** requests the NameNode to allocate new blocks and choose DataNodes to store replicas (typically 3 DataNodes for a replication factor of 3).
5. A pipeline is formed between the selected DataNodes (e.g., DataNode A → DataNode B → DataNode C). The Data Streamer streams packets to the first DataNode, which passes them to the second, which passes them to the third.
6. When the client finishes writing data, it calls 'close()', which flushes all remaining packets to the DataNodes and contacts the NameNode to commit the file creation.

5. Data Ingestion: Sqoop and Flume

To analyze data in Hadoop, it must first be ingested from external sources.

5.1 Apache Sqoop

Apache Sqoop (SQL to Hadoop)

Sqoop is a tool designed to transfer bulk data between relational databases (RDBMS) and the Hadoop Ecosystem (HDFS, Hive, HBase). It translates import and export commands into MapReduce jobs to execute parallel data transfers.

- **Import Mechanism:** Reads rows from an RDBMS table, generates a MapReduce job where mappers read data in parallel using SQL boundaries, and writes the output files to HDFS.
- **Export Mechanism:** Reads files from HDFS and inserts records into a target database table in parallel using database insert statements.

5.2 Apache Flume

Apache Flume

Flume is a distributed service designed for efficiently collecting, aggregating, and moving large amounts of streaming log data (such as web server logs or social media streams) into HDFS.

An active Flume agent consists of three main components:

1. **Source:** Receives event data from external generators (e.g., Syslog, HTTP) and pushes it into one or more channels.

2. **Channel:** A passive store that buffers the events until they are consumed by a sink (can be in-memory for speed or file-based for durability).
3. **Sink:** Removes events from the channel and stores them in an external repository like HDFS or HBase.

6. Hadoop Archives (HAR)

Every file, directory, and block in HDFS occupies about 150 bytes of memory in the NameNode's RAM. If a cluster stores millions of small files (smaller than the block size), the NameNode RAM becomes a performance bottleneck.

Hadoop Archives (HAR)

Hadoop Archives (HAR) are file archiving tools that pack HDFS files into larger archive files ('.har' files) to reduce the metadata overhead in the NameNode, while still allowing applications to access the individual files transparently.

A '.har' archive contains index files listing the metadata and a set of part files containing the concatenated data blocks.

7. Hadoop I/O: Compression, Serialization, and File Structures

7.1 Compression Codecs

Hadoop supports data compression to save storage space and reduce network and disk I/O overhead.

Codec	Algorithm	Splittable?	Key Characteristics
Gzip	DEFLATE	No	High compression ratio, slow speed.
Bzip2	Burrows-Wheeler	Yes	Extremely high compression ratio, very slow.
LZO	LZO	Yes (if indexed)	Fast compression and decompression.
Snappy	Snappy	No	Very fast speed, moderate compression.

Why Splittability Matters: If a compression format is splittable, MapReduce can split the file and process different portions on separate mappers in parallel. If it is not splittable, a single mapper must process the entire file sequentially, breaking the parallel execution model.

7.2 Serialization

Serialization

Serialization is the process of converting structured objects into a stream of bytes for transmission over a network or storage on disk. **Deserialization** is the reverse process of turning a byte stream back into live runtime objects.

Hadoop does not use standard Java serialization because it is too bloated. Instead, it defines its own lightweight protocols:

- **Writable Interface:** Mappers and Reducers exchange keys and values implementing the **Writable** interface. It provides high-speed serialization using compact byte formats.
- **Avro:** A language-neutral serialization system that stores data schemas in JSON files, making it easy for different languages (Java, C++, Python) to share data records.

7.3 File-Based Data Structures

Hadoop provides specialized container files for binary data: 1. **SequenceFile:** A persistent, binary container file format for storing key-value pairs. It supports compression and includes **sync markers** to allow MapReduce splits. 2. **MapFile:** A sorted version of **SequenceFile** that contains an index file ('/index') alongside the data file ('/data') to allow fast random access by key.

Quick Revision — Module 2

HDFS Concepts, CLI & I/O Architectures | OE-EC803B

One-Line Definitions

- **DataNode:** A slave node in HDFS that physically stores data blocks on local disks.
- **Secondary NameNode:** A helper node that periodically merges Edit Logs with FSImage (Checkpointing).
- **Splittability:** The property of a file format/codecs that allows it to be split into chunks for parallel MapReduce execution.
- **SequenceFile:** A persistent binary file format that stores sequential key-value data.
- **Avro:** A serialization framework using JSON-defined schemas for cross-language data exchanges.

Frequently Asked / Viva Questions

Q1. Why does HDFS use a large block size (e.g., 128MB)?

Large block sizes minimize the disk seek overhead. Since disk seeks take several milliseconds, setting the block size to 128MB ensures that the disk transfer time (reading data) is significantly larger than the seek time, maximizing throughput.

Q2. How does checkpointing work in the Secondary NameNode?

The Secondary NameNode fetches the active Edit Log and current FSImage from the NameNode, merges them in memory to generate a new FSImage checkpoint, and sends it back to the NameNode. This keeps the Edit Log from growing too large.

Q3. Describe the HDFS replication strategy (Rack Awareness).

For a replication factor of 3: HDFS places the first replica on a node in the local rack, the second replica on a different node in a remote rack, and the third replica on a different node within that same remote rack. This balances fault tolerance and write latency.

Q4. What happens if a DataNode fails in an HDFS cluster?

If a DataNode fails to send Heartbeats to the NameNode for a timeout period (typically 10 minutes), the NameNode marks it as dead. It scans the metadata, identifies all blocks stored on the failed node, and schedules copies of those blocks to other nodes to restore the replication factor.

Q5. What is the difference between client-side reads and client-side writes in HDFS?

During a read, the client only queries the NameNode for block locations and reads directly from the nearest DataNode. During a write, the client streams packets to a pipeline of DataNodes, which replicate data to each other while sending ACKs back.

Frequently Asked / Viva Questions (contd.)

Q6. Explain the agent architecture of Apache Flume.

A Flume agent consists of a **Source** (ingests raw logs from external clients), a **Channel** (buffers events in memory or on disk), and a **Sink** (drains the channel to write files into HDFS).

Q7. Why are Hadoop Archives (HAR) used for small files?

Each file in HDFS consumes 150 bytes of metadata in the NameNode's RAM. Millions of small files can exhaust NameNode memory. HAR bundles these files into larger archive containers, reducing the total metadata footprint.

Q8. Which compression codecs are splittable in MapReduce?

Bzip2 is natively splittable. LZO is splittable if an index is built for it. Gzip and Snappy are not splittable.

Q9. Why did Hadoop develop its own Writable serialization system instead of using Java Serialization?

Java's default serialization is too heavy, storing class names, parent classes, and metadata along with the data. Hadoop's Writable system is lightweight, compact, and optimized for high-speed network transmission.

Q10. What is the difference between a SequenceFile and a MapFile?

A SequenceFile stores binary key-value pairs sequentially. A MapFile is a directory containing a sorted SequenceFile ('/data') and an index file ('/index') that maps keys to offsets for random lookups.

Comparison Snapshot

NameNode vs. DataNode:

Feature	NameNode	DataNode
Role	Master Node	Slave Node
Primary Data stored	File metadata, namespace mapping, block locations	Actual data blocks (replicated payload)
Storage Media	System RAM (for speed)	Physical Hard Drive / SSD
Hardware requirements	Enterprise grade (high RAM capacity)	Commodity servers (large storage disks)
Quantity in Cluster	Active/Standby pair (HA configurations)	Thousands of nodes in large clusters
Failure Impact	High (cluster down if master fails without HA)	Low (re-replicated from other nodes)

Module 3

Anatomy of a MapReduce Job Run

1. Anatomy of a MapReduce Job Run

MapReduce execution is divided into MapReduce 1 (Classic MapReduce) and MapReduce 2 (built on YARN).

Classic MapReduce vs. YARN

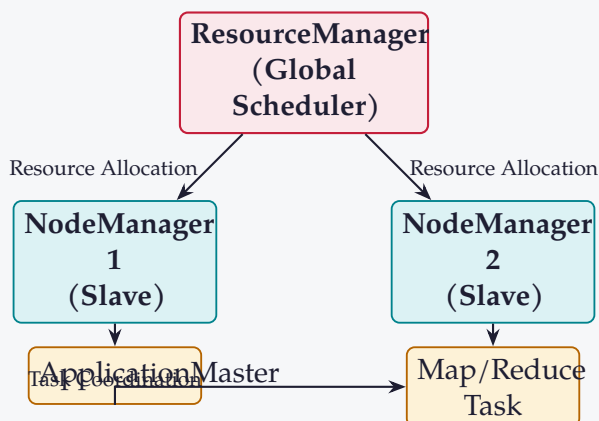
- **MapReduce 1 (Classic):** Job execution was coordinated by a single master process called the **JobTracker**, which scheduled tasks and monitored slaves called **TaskTrackers**.
- **YARN (MapReduce 2):** Divides the resource management and job scheduling functions into separate daemons: a global **ResourceManager** and application-specific **ApplicationMasters**.

1.1 YARN Runtime Job Execution Steps

YARN runs MapReduce jobs using several steps:

1. **Job Submission:** The Client program calls 'submitJob()' on the Job object, requesting a new Application ID from the ResourceManager.
2. **Application Master Launch:** The ResourceManager allocates a **Container** (allocated CPU/RAM limit) on a NodeManager and starts the **ApplicationMaster** process there.
3. **Resource Allocation:** The ApplicationMaster registers with the ResourceManager and requests containers across NodeManagers to host the Map and Reduce tasks.
4. **Task Execution:** The ApplicationMaster contacts the NodeManagers to launch the tasks within the allocated containers. Tasks retrieve data blocks from local HDFS DataNodes, process them, and write outputs locally.

YARN Resource Management Layout



2. Failures in YARN

Distributed systems must tolerate node and task failures gracefully.

2.1 Task Failure

A Map or Reduce task can fail due to runtime Java exceptions or JVM crashes.

- **Detection:** The NodeManager detects that the task JVM exited with a non-zero code and reports the failure back to the ApplicationMaster.
- **Resolution:** The ApplicationMaster reschedules the task on a different NodeManager. It will retry the task up to 4 times (default limit) before marking the entire job as failed.

2.2 NodeManager Failure

- **Detection:** The NodeManager sends heartbeats to the ResourceManager. If heartbeats stop (typically for 10 minutes), the ResourceManager marks the NodeManager as dead.
- **Resolution:** The ResourceManager notifies the active ApplicationMasters. The ApplicationMasters reschedule all tasks (both active and completed map tasks) that were running on the failed NodeManager, since their local map outputs are no longer accessible.

2.3 ResourceManager / ApplicationMaster Failure

- **ApplicationMaster Failure:** The ResourceManager detects the loss of heartbeat from the ApplicationMaster and starts a new container to launch a fresh ApplicationMaster instance.
- **ResourceManager Failure:** This is a single point of failure (SPOF). It is resolved by running active-standby ResourceManagers using ZooKeeper to manage failover and state replication.

3. Job Scheduling

YARN supports three primary job scheduling engines:

1. **FIFO (First-In, First-Out) Scheduler:** Allocates resources strictly based on job submission order. A large, slow job at the head of the queue will block all subsequent small jobs.
2. **Capacity Scheduler:** Allocates distinct organizational queues, where each queue is guaranteed a fixed percentage of cluster capacity (e.g., Queue A gets 40%, Queue B gets 60%). If a queue has free space, it can temporarily lend it to other queues.
3. **Fair Scheduler:** Dynamically balances resources so that all running jobs get an equal share of cluster capacity over time. When a single job runs, it uses 100% of the cluster; when a second job is submitted, YARN re-allocates 50% of the resources to it.

4. Shuffle and Sort Phase

The Shuffle and Sort

The **Shuffle and Sort** is the core transport layer of MapReduce that routes intermediate key-value outputs from mappers to the correct reducers, guaranteeing that all values associated with a specific key are grouped and delivered to the same reducer in sorted order.

The shuffle process runs across the map and reduce stages:

4.1 Map Side Execution Details

1. **Circular Buffer:** The Map task writes outputs to an in-memory circular RAM buffer (default size: 100MB).
2. **Spilling:** When the buffer fill percentage reaches a threshold (default: 80%), a background thread wakes up to spill the data to the local disk.
3. **Partitioning:** Before writing to disk, the thread divides the key-value records into partitions based on the target Reducer ID (using `HashPartitioner` by default: `(key.hashCode() & Integer.MAX_VALUE) % numReducers`).
4. **Sorting:** Within each partition, the keys are sorted in memory using a quicksort algorithm.
5. **Combining:** If a local 'Combiner' (mini-reducer) is defined, it runs on the sorted partition records before writing to disk to reduce data volume.
6. **Merging:** When the Map task finishes, all local spill files are merged and sorted to create a single partitioned and sorted output file on the local node.

4.2 Reduce Side Execution Details

1. **Copy Phase:** The Reducer tasks have copier threads that fetch their respective partition files from all mapper nodes over HTTP (using the `MapOutputServlet`).

2. **Merge Phase:** As files are copied, background threads merge and sort the incoming data in passes.
3. **Reduce Phase:** The sorted key-value pairs are fed to the 'reduce()' method line-by-line.

5. MapReduce Types and Formats

MapReduce enforces strict type checking using serialization wrappers (e.g., 'Text' for Java String, 'IntWritable' for Integer).

5.1 Input Splits and InputFormat

- **InputSplit:** A chunk of HDFS data processed by a single mapper. Typically equals the HDFS block size (128MB).
- **InputFormat:** Validates the input files and defines how to split them. It spawns the **RecordReader** to convert split byte streams into key-value records.

InputFormat	Key Type	Value Type
'TextInputFormat'	'LongWritable' (Byte offset of line)	'Text' (Content of the text line)
'KeyValueTextInputFormat'	'Text' (Text before separator)	'Text' (Text after separator)
'SequenceFileInputFormat'	User-defined binary key type	User-defined binary value type

6. Advanced MapReduce Features

6.1 Counters

Counters are used to collect statistics about MapReduce jobs (e.g., counting empty records or corrupt lines).

- **Built-in Counters:** Automatically populated by YARN (e.g., Map Input Records, Bytes Written).
- **User-Defined Counters:** Programmed in Java using enums:

```
context.getCounter(MyEnum.CORRUPT_ROWS).increment(1);
```

6.2 Joins in MapReduce

When combining datasets (e.g., Customers and Orders tables), two join strategies are used:

Feature	Map-Side Join	Reduce-Side Join
Execution Phase	Done completely inside the Mapper.	Done inside the Reducer.
Prerequisites	Datasets must be sorted and bucketed by the join key. One dataset must be small enough to fit in memory.	None. General-purpose join.
Shuffle Overhead	Zero shuffle overhead (fastest).	Heavy network shuffle of all records.
Data Volume Limit	Limited by Mapper node RAM.	Unlimited (handles massive datasets).

Quick Revision — Module 3

MapReduce Internals, YARN Scheduler & Joins | OE-EC803B

One-Line Definitions

- **ApplicationMaster:** A per-application YARN process that negotiates resources and coordinates task runs.
- **Container:** A logical YARN resource allocation containing CPU cores and RAM limits on a NodeManager.
- **Speculative Execution:** A task execution strategy that runs duplicate copies of slow-running tasks in parallel to minimize job latency.
- **Combiner:** A local semi-reducer that aggregates Map outputs locally before the network shuffle.
- **RecordReader:** A helper class that translates raw HDFS data splits into structured key-value objects.

Frequently Asked / Viva Questions

Q1. What are the differences between MapReduce 1 (Classic) and YARN?

In MapReduce 1, the JobTracker was responsible for both resource management and task scheduling, creating scalability bottlenecks. YARN decouples these tasks, using a global **ResourceManager** for cluster resource allocation and a per-job **ApplicationMaster** for scheduling.

Q2. Detail the spilling process on the Map side.

Mappers write outputs to a 100MB circular buffer. When the buffer is 80% full, a background thread locks the data, partitions it based on Reducer IDs, sorts it by key, runs the optional combiner, and writes (spills) the sorted partition files to local disk.

Q3. How does the Fair Scheduler distribute cluster resources?

The Fair Scheduler divides resources dynamically so that all active jobs receive an equal share of CPU and RAM over time. If only one job runs, it uses the entire cluster. When a second job starts, YARN re-allocates 50% of the resources to it.

Q4. Why does a NodeManager failure force mappers to rerun, but not reducers?

Mappers store their intermediate partition outputs on local disks, so if the mapper's node dies, those files are lost. Reducers write their final outputs directly to replicated HDFS blocks, which remain accessible even if a NodeManager fails.

Q5. What is the difference between TextInputFormat and KeyValueTextInputFormat?

'TextInputFormat' treats the line offset as the key ('LongWritable') and the line text

Frequently Asked / Viva Questions (contd.)

as the value ('Text'). 'KeyValueTextInputFormat' splits each line at a separator character (like a tab) to define both the key and the value as 'Text'.

Q6. Explain Map-side Joins and their memory constraints.

A Map-side Join joins datasets inside the Mapper without a network shuffle. It requires that all datasets are pre-sorted and bucketed by the join key, and one of the datasets must be small enough to load entirely into the mapper's JVM memory.

Q7. Why does speculative execution not run for tasks with side effects?

Speculative execution launches duplicate tasks. If a task writes to an external database, running duplicate tasks would cause duplicate writes. Therefore, speculative execution should be disabled for tasks with external side effects.

Q8. What is the purpose of sync markers in a SequenceFile?

Sync markers are special byte sequences placed periodically in a SequenceFile. They allow MapReduce to split the file at arbitrary boundaries, as the RecordReader can seek to a random position and scan forward to find the next sync marker.

Q9. How does secondary sorting work in MapReduce?

It allows sorting the values associated with a key. It is implemented by creating a composite key (comprising the natural key and the secondary sort value), partitioning by the natural key, and sorting by the composite key.

Q10. What does the HashPartitioner do?

It computes the partition index for a mapper output key: `(key.hashCode() & Integer.MAX_VALUE) % numReducers`. This ensures that all records with the same key are routed to the same Reducer ID.

Comparison Snapshot**Capacity Scheduler vs. Fair Scheduler:**

Feature	Capacity Scheduler	Fair Scheduler
Primary Design Goal	Organization queues with guaranteed shares	Dynamic, equal sharing of cluster resources
Queue Configuration	Strict hierarchical queues (e.g., Sales, Dev)	Simple resource shares per running application
Resource Loaning	Queues can borrow unused capacity from others	Idle capacity is divided among active jobs
Preemption	Not supported (waits for tasks to end)	Supported (reclaims resources by killing tasks)
Use Case	Multi-tenant corporate cluster with fixed budgets	Ad-hoc environments with variable workloads

Module 4

Apache Pig

1. Apache Pig

Apache Pig

Apache Pig is a high-level dataflow platform designed for processing large datasets on Hadoop. It provides a procedural dataflow programming language called **Pig Latin**, which compiles scripts into sequences of MapReduce jobs executed across a YARN cluster.

1.1 Execution Modes and Components

- **Local Mode:** Runs in a single JVM using the local filesystem. Excellent for testing and debugging.
- **MapReduce Mode:** Translates scripts into MapReduce jobs and executes them on an active Hadoop cluster.
- **Grunt Shell:** The interactive command-line shell used to type Pig Latin statements.
- **UDF (User Defined Functions):** Allows developers to write custom processing logic in Java, Python, or JavaScript when Pig's built-in operators are insufficient.

1.2 Pig Latin Data Model

Pig Latin supports both simple types (e.g., 'int', 'chararray') and complex nested types:

- **Atom:** Any single value stored as a string (e.g., "Biraj", '45').
- **Tuple:** An ordered set of fields (e.g., '(Biraj, 22, CGEC)').
- **Bag:** A collection of tuples (e.g., '(Biraj, 22), (Amit, 23)').
- **Map:** A set of key-value pairs (e.g., ['name' #'Biraj', 'age' #'22']).

1.3 Core Processing Operators

- **'LOAD':** Ingests data from HDFS files.
- **'FILTER':** Selects tuples from a relation based on conditions.
- **'FOREACH ... GENERATE':** Transforms data columns.

- 'GROUP': Collects tuples sharing a key into a bag.
- 'JOIN': Joins relations based on matching keys.

2. Apache Hive

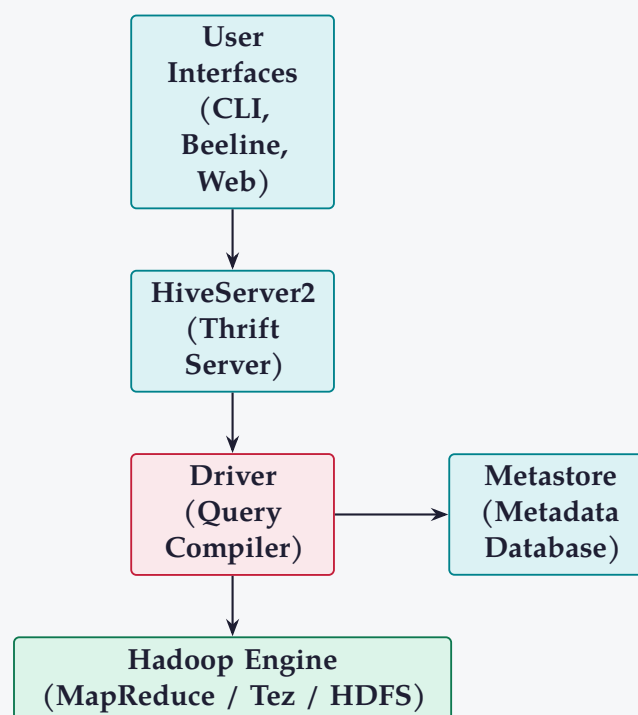
Apache Hive

Apache Hive is a data warehouse software system built on top of Apache Hadoop that provides data summarization, ad-hoc querying, and analysis of large datasets stored in HDFS. It provides an SQL-like query language called **HiveQL**.

2.1 Hive Architecture

Hive translates SQL-like HiveQL statements into MapReduce, Tez, or Spark jobs.

Hive System Architecture



- **Metastore:** A relational database (typically MySQL or Derby) that stores schema definitions, column details, and partition metadata.
- **HiveServer2:** The execution endpoint allowing remote JDBC/ODBC clients to submit queries.

2.2 Managed vs. External Tables

Hive supports two types of tables to control data ownership:

Feature	Managed Table (Internal)	External Table
Data Ownership	Hive owns both the metadata and the data.	Hive only owns the metadata.
Storage Path	Default directory: '/user/hive/warehouse/'	User-defined HDFS location.
'DROP TABLE' Impact	Deletes both the HDFS data and metadata.	Deletes metadata; HDFS data remains intact.
Use Case	Temporary tables, internal application data.	Sharing datasets with Spark, Pig, or MapReduce.

2.3 Partitioning vs. Bucketing

Hive optimizes query performance using two layout patterns:

Partitioning vs. Bucketing Properties

- **Partitioning:** Divides a table into subdirectories based on column values (e.g., '/country=IN/', '/country=US/'). It significantly reduces query search space by pruning directories.
- **Bucketing:** Distributes table rows into a fixed number of files (buckets) based on a hash function of a column (e.g., 'hash(userID) % numBuckets'). It speeds up joins and sampling on high-cardinality columns.

3. Apache HBase

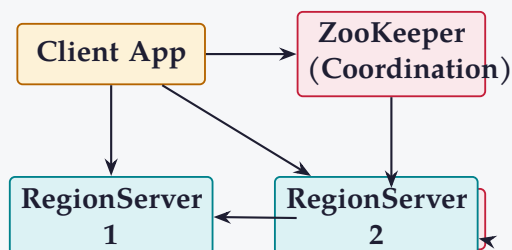
Apache HBase

Apache HBase is a distributed, column-oriented, multi-dimensional, NoSQL database built on top of HDFS, modeled after Google's Bigtable, designed to provide random, real-time read/write access to billions of rows and columns.

3.1 HBase Architecture and Components

HBase achieves horizontal scale using master-slave coordinators:

- **HMaster:** Coordinates the cluster, assigns regions to RegionServers, and handles schema updates.
- **RegionServer:** Manages a set of dynamic table splits called **Regions**. Handles client read/write requests directly.
- **Write-Ahead Log (WAL):** A local disk file where writes are written first for durability before committing to RAM.
- **MemStore:** The write buffer in RAM. Writes are accumulated here first.
- **HFile:** The physical file format stored on HDFS. When a MemStore fills up (flushes), its contents are written to disk as an HFile.

HBase Architecture

4. IBM Big SQL

IBM Big SQL

IBM Big SQL is an enterprise-grade, SQL-on-Hadoop engine that provides an ANSI-SQL compliant query compiler and Massively Parallel Processing (MPP) execution framework to query data stored across HDFS, Hive, and HBase directly.

Big SQL utilizes a shared-nothing MPP architecture that executes SQL queries natively on Hadoop data blocks without the latency overhead of MapReduce translations, supporting complex joins and standard business intelligence integrations.

Quick Revision — Module 4

Pig Latin, HiveQL, HBase NoSQL & Big SQL | OE-EC803B

One-Line Definitions

- **Pig Latin:** A high-level dataflow programming language used to describe data transformation pipelines.
- **HiveQL:** An SQL-like query language that is compiled into execution plans (like Tez or MapReduce).
- **Metastore:** The repository that stores structural metadata and schema information for Hive tables.
- **RegionServer:** An HBase slave node that serves read and write operations for a subset of row keys.
- **Big SQL:** An enterprise Massively Parallel Processing (MPP) SQL querying engine for Hadoop clusters.

Frequently Asked / Viva Questions

Q1. Compare Apache Pig and Apache Hive.

Apache Pig uses a procedural dataflow scripting language (Pig Latin) suited for ETL pipelines and data engineers. Apache Hive uses a declarative SQL-like language (HiveQL) suited for data warehousing and business intelligence.

Q2. What happens to the HDFS data when a Managed Table vs. an External Table is dropped in Hive?

Dropping a Managed Table deletes both the table metadata and the physical data files in HDFS. Dropping an External Table deletes only the Hive schema metadata, leaving the physical data files intact in HDFS.

Q3. Explain the difference between Partitioning and Bucketing in Hive.

Partitioning organizes data into separate subdirectories based on column values (pruning queries). Bucketing splits data into a fixed number of files based on a hash of a column, which optimizes joins on high-cardinality columns.

Q4. Detail the write path of a record in Apache HBase.

A client write is first appended to the Write-Ahead Log (WAL) on disk for durability, then written to the in-memory **MemStore** in RAM. Once the write is in MemStore, the client receives an ACK.

Q5. When does HBase flush data, and what file format is generated?

When the in-memory MemStore reaches its capacity threshold (typically 128MB), HBase flushes its sorted key-value contents to HDFS, creating a new binary **HFile**.

Q6. Why is HBase called a Column-Family NoSQL database?

Frequently Asked / Viva Questions (contd.)

HBase groups columns together into **Column Families** (defined during table creation). Internally, columns belonging to the same column family are stored in the same physical HFile, optimizing access for selective queries.

Q7. What are the complex nested data types supported in Pig Latin?

Tuple (an ordered list of fields), **Bag** (an unordered collection of tuples), and **Map** (a set of key-value pairs).

Q8. Explain the role of ZooKeeper in an HBase cluster.

ZooKeeper coordinates Region assignment, detects RegionServer failures, maintains the root table location, and prevents active-standby conflicts by coordinating HMaster elections.

Q9. What are UDF, UDAF, and UDTF in Hive?

- **UDF:** User Defined Function (maps 1 row to 1 output).
- **UDAF:** User Defined Aggregate Function (maps multiple rows to 1 aggregated output).
- **UDTF:** User Defined Table Generating Function (maps 1 row to multiple output rows/tables).

Q10. Describe the execution flow of a Big SQL query.

Big SQL bypasses MapReduce. The Big SQL coordinator compiles SQL queries, optimizes execution plans, and uses an MPP engine to distribute query processing fragments directly to local nodes containing target HDFS blocks.

Comparison Snapshot

Traditional RDBMS vs. NoSQL HBase:

Feature	Traditional RDBMS	NoSQL HBase
Data Layout	Row-oriented (stores entire row together)	Column-family oriented (stores family columns together)
Schema Flexibility	Schema-on-write (rigid definitions)	Schema-less (columns can be added dynamically)
Transaction Support	Full ACID transactions across multiple tables	ACID transactions supported at Row-level only
Scaling Pattern	Vertical (bigger machines)	Horizontal (adding RegionServers in parallel)
Join Support	Rich declarative SQL joins	No native join support (requires MapReduce/Spark)
Null Value Storage	Consumes space for NULL definitions	Zero storage overhead for empty cells (sparse tables)

Module 5

R and Machine Learning in Big Data

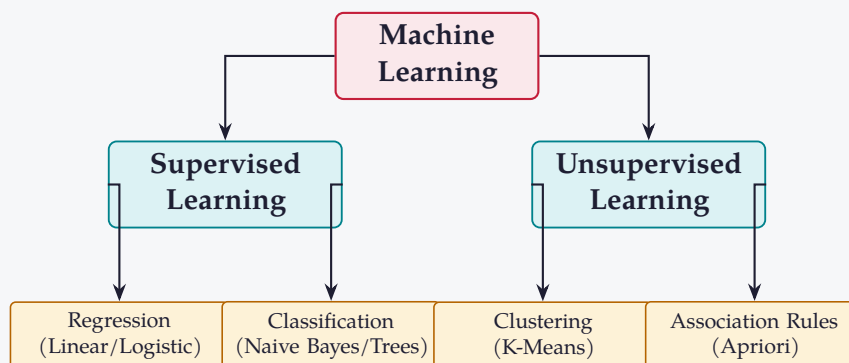
1. R and Machine Learning in Big Data

Data analytics turns raw database bits into business value. R is a popular programming language for statistical computing and graphics. In Big Data architectures, statistical models are divided into supervised and unsupervised techniques:

Machine Learning Paradigms

- **Supervised Learning:** Training models using labeled datasets where the input features (X) are mapped to a known target variable (Y).
- **Unsupervised Learning:** Identifying hidden patterns, groups, or structures in unlabeled datasets where only input features (X) are provided.

Taxonomy of Machine Learning



2. Supervised Learning Algorithms

2.1 Linear Regression

Used to predict a continuous numeric output variable (Y) from one or more input features (X).

Linear Regression Modeling

The prediction hypothesis is modeled as:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n = \theta^T x$$

The model parameters (θ_i) are calculated by minimizing the **Mean Squared Error (MSE) Cost Function**:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Parameters are updated iteratively using the **Gradient Descent** rule:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(where α is the learning rate parameter).

2.2 Logistic Regression

Despite its name, it is a classification algorithm used to predict binary categorical outcomes ($Y \in \{0, 1\}$).

Sigmoid Function

The prediction is modeled by feeding the linear hypothesis into the **Sigmoid (Logistic) Function** to output a probability between 0 and 1:

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

- If $h_{\theta}(x) \geq 0.5 \Rightarrow$ Predict 1
- If $h_{\theta}(x) < 0.5 \Rightarrow$ Predict 0

2.3 Decision Trees

A non-parametric model that builds a tree hierarchy by splitting datasets based on features that maximize purity:

- **Entropy:** Measures the impurity or randomness of a dataset:

$$H(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

- **Information Gain:** Measures the reduction in entropy after splitting on a feature A :

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

2.4 Naive Bayes Classification

A probabilistic classification algorithm based on Bayes' Theorem:

Bayes' Theorem and Naive Assumption

$$P(C_k | x) = \frac{P(x | C_k)P(C_k)}{P(x)}$$

Naive conditional independence assumption: Assumes that the presence of a particular feature in a class is completely independent of the presence of any other feature:

$$P(x_1, x_2, \dots, x_n | C_k) = \prod_{i=1}^n P(x_i | C_k)$$

Combining these yields the Naive Bayes classification rule:

$$\hat{y} = \arg \max_k P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

3. Unsupervised Learning Algorithms

3.1 K-Means Clustering

An iterative clustering algorithm that partitions m data points into K distinct, non-overlapping groups (clusters):

1. **Initialize:** Randomly choose K centroids $(\mu_1, \mu_2, \dots, \mu_K)$.
2. **Assign:** Group each data point $x^{(i)}$ to its closest centroid:

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2$$

3. **Update:** Recompute the centroid values as the mean of all data points assigned to that cluster:

$$\mu_j := \frac{\sum_{i=1}^m \mathbb{I}(c^{(i)} = j) x^{(i)}}{\sum_{i=1}^m \mathbb{I}(c^{(i)} = j)}$$

4. **Repeat:** Repeat steps 2 and 3 until centroids stop changing.

3.2 Association Rule Mining: Apriori Algorithm

Used to identify relationships between variables in large transaction databases (e.g., market basket analysis).

Association Rule Metrics

An association rule is written as $X \rightarrow Y$ (e.g., $\{Bread\} \rightarrow \{Butter\}$).

- **Support:** The fraction of total transactions containing both itemsets:

$$Support(X \rightarrow Y) = \frac{Count(X \cup Y)}{N}$$

- **Confidence:** How often items in Y appear in transactions containing X :

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- **Lift:** The ratio of the rule's confidence to its expected confidence under independence:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

(If $\text{Lift} > 1$, items are highly dependent; if $\text{Lift} < 1$, they are substitutes).

The **Apriori Algorithm** uses the downward closure property: *all subsets of a frequent itemset must also be frequent*. If an itemset is infrequent, all its supersets are pruned immediately.

4. Collaborative Filtering

Collaborative Filtering

Collaborative Filtering is a recommendation system technique that filters information or patterns using the preferences or historical behaviors of multiple users, based on the assumption that users who agreed in the past will agree in the future.

Two main collaborative filtering patterns are:

1. **User-Based Collaborative Filtering:** Identifies users similar to the active user (based on rating history) and recommends items that those similar users liked.
2. **Item-Based Collaborative Filtering:** Calculates similarities between items based on user ratings. Recommends items similar to those the active user rated highly in the past.

Cosine Similarity Metric

The similarity between two user rating vectors (or item rating vectors) A and B is measured using **Cosine Similarity**:

$$\text{Similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_i A_i B_i}{\sqrt{\sum_i A_i^2} \sqrt{\sum_i B_i^2}}$$

5. Big Data Analytics with BigR

Standard R loads all data structures entirely into the local machine's RAM. If a dataset is 1TB, native R will crash with out-of-memory errors.

BigR

BigR is an enterprise R package that provides a client-side interface connecting R sessions directly to IBM InfoSphere BigInsights or Apache Spark clusters, allowing users to execute statistics and machine learning on distributed datasets in parallel.

BigR uses **deferred execution** to translate R commands into MapReduce or SQL operations that run directly on HDFS data blocks, returning only the final, aggregated results back to the user's local R terminal.

Quick Revision — Module 5

Machine Learning, Association Rules & BigR | OE-EC803B

One-Line Definitions

- **Supervised Learning:** Training models using input-output pairs to predict targets for unseen data.
- **Unsupervised Learning:** Finding patterns in data without using target labels.
- **Entropy:** A thermodynamic-inspired metric measuring the impurity of a dataset split.
- **Collaborative Filtering:** Making recommendations by comparing user rating histories.
- **BigR:** An R package that runs parallel analytical calculations on distributed Hadoop clusters.

Frequently Asked / Viva Questions

Q1. What is the difference between Linear and Logistic Regression?

Linear regression predicts continuous numeric variables (e.g., salary, price) by fitting a straight line. Logistic regression classifies binary categorical variables (e.g., spam/not-spam) by passing a linear hypothesis through the sigmoid function to output probabilities.

Q2. Detail the mathematical steps of Gradient Descent.

It starts with random parameter values (θ). In each iteration, it computes the gradient (partial derivative) of the Cost Function $J(\theta)$ with respect to each parameter, and updates the parameters in the direction of the steepest descent: $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$.

Q3. Explain the role of Entropy in Decision Tree splits.

Entropy measures the impurity of a node: $H(S) = -\sum p_i \log_2 p_i$. The decision tree algorithm calculates the entropy of the current node, splits the data using a feature, and computes the entropy of the child nodes. It selects the feature that maximizes the reduction in entropy (Information Gain).

Q4. Why is Naive Bayes called “Naive”?

Because it assumes that all input features are conditionally independent of each other given the class label. In reality, features are often correlated (e.g., height and weight). Despite this assumption, it works well in practice for text classification.

Q5. How does the K-Means clustering algorithm converge?

It converges when the assignments of data points to clusters stop changing, or when the positions of the cluster centroids remain constant across iterations, minimizing the total within-cluster sum of squared errors.

Frequently Asked / Viva Questions (contd.)

Q6. What is the difference between Support and Confidence in the Apriori algorithm?

Support is the percentage of all transactions that contain the itemset ($Support(X \cup Y)$). Confidence is the conditional probability that a transaction contains Y given that it contains X ($Support(X \cup Y)/Support(X)$).

Q7. Why does the Apriori algorithm prune infrequent itemsets?

Because of the Apriori property: *if an itemset is frequent, all its subsets must be frequent*. Conversely, if an itemset is infrequent, all its supersets must also be infrequent, allowing them to be pruned immediately.

Q8. Compare User-Based and Item-Based Collaborative Filtering.

User-Based filtering calculates similarities between users to recommend items. It scales poorly because user preferences change frequently. Item-Based filtering calculates similarities between items. Since item attributes are static, these similarity matrices can be computed offline, making it more scalable.

Q9. How is Cosine Similarity used in recommendation engines?

It measures the similarity between two rating vectors by calculating the cosine of the angle between them: $\frac{A \cdot B}{\|A\| \|B\|}$. A value of 1 indicates identical rating patterns, while 0 indicates orthogonal (dissimilar) patterns.

Q10. Why is native R unsuited for Big Data, and how does BigR resolve this?

Native R loads all data structures directly into memory, which crashes on large datasets. BigR acts as an API gateway, translating R commands into SQL or MapReduce operations that run directly on HDFS, avoiding memory bottlenecks.

Comparison Snapshot

Supervised vs. Unsupervised Machine Learning:

Feature	Supervised Learning	Unsupervised Learning
Input Data	Labeled training datasets (features + target labels)	Unlabeled datasets (features only)
Primary Goal	Predict target outputs (Y) for new, unseen inputs (X)	Uncover hidden structures, patterns, or groupings
Evaluation	Metrics: Accuracy, Precision, Recall, MSE, R^2	Metrics: Silhouette score, Inertia, Lift
Algorithms	Linear/Logistic Regression, Naive Bayes, Decision Trees	K-Means Clustering, Apriori, Principal Component Analysis
Complexity	High supervision required (data labeling is expensive)	Low supervision; patterns are discovered automatically